

Subroutines and Control Abstraction

The background of the slide features a gradient of blue at the top, transitioning into a series of stylized, overlapping waves in shades of yellow and light blue that sweep across the lower half of the image.

Outline

- Tail Recursion
- Tree Recursion
- Stack Smashing

Tail Recursion

- A tail-recursive function
 - Is one in which additional computation never follows a recursive call, i.e., the return value is simply whatever the recursive call returns.
- Dynamically allocated stack space is unnecessary
- The compiler can reuse the space belonging to the current iteration when it makes the recursive call.

```
// Inherent Tail-Recursive Function
int gcd(int m, int n) {
    if(n==0) return m;
    return gcd(n, m%n); }
```

Tail Recursion Example

6, 0	
12, 6	
18, 12	
30, 18	
param x	

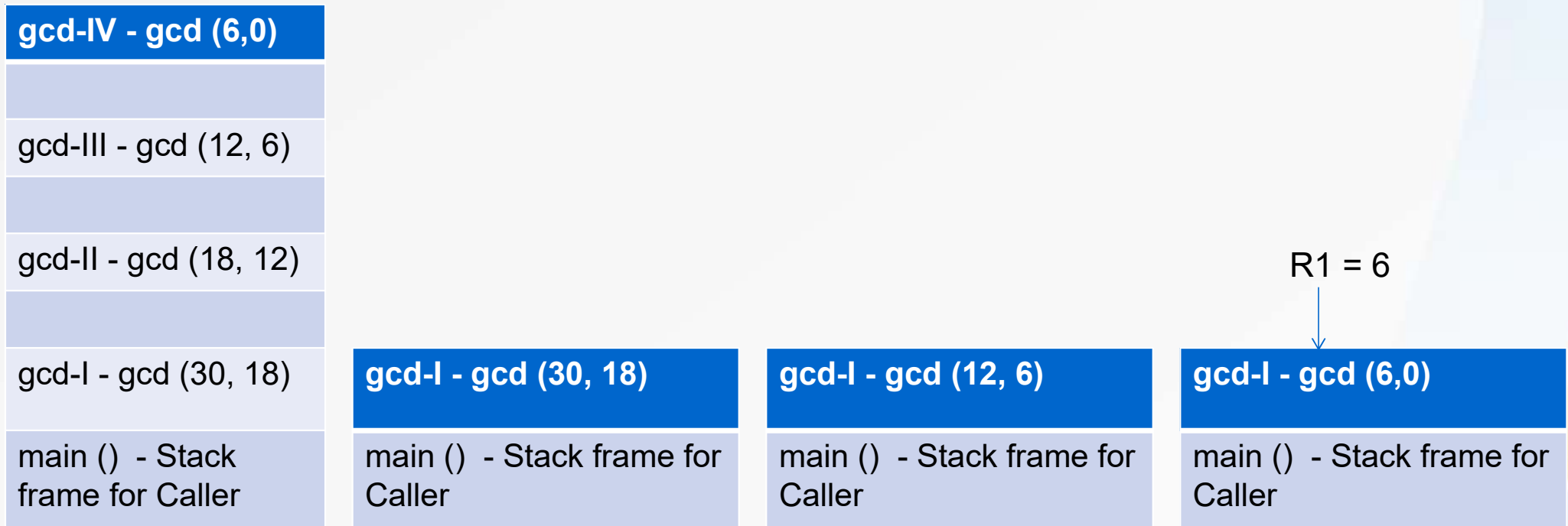
gcd (6, 0)
gcd (12, 6)
gcd (18, 12)
gcd (30, 18)
main()

—————→ R1 = 6

```
int gcd(int m, int n)
{
    if(n==0) return m;
    return gcd(n, m%n); }
```

```
x = gcd(30,18)
```

Tail Recursion Example Cont..



Stack frame popped out for every call and the same space is reallocated for a frame of the next recursive call

Recursive to Tail-Recursive Function

- **Non-tail Recursive Function**

- ```
int fact(unsigned int n) {
 if (n == 0) return 1;
 return n*fact(n-1); }
```
- The function is not tail-recursive because the value returned by `fact(n-1)` is used in `fact(n)` and call to `fact(n-1)` is not the last thing done by `fact(n)`.
- Can we convert a recursive function to a tail-recursive function?
- Can be done using a jacket function for every recursive call/function.

- **Tail Recursive Function**

- ```
int factorial (int n) { return fac (n,1) }  
int fac(int n, int a)  
{  
    if (n==0) return a;  
    return fac(n-1, n*a);  
}
```
- what is use of 'a'?
 - 'a' (accumulator) would be an extra parameter used for the purpose of storing the partial results.

Recursive to Tail-Recursive Function Cont..

```
int factorial (int n) {  
    return fac (n,1) }
```

```
int fac(int n, int a) {  
    if (n==0) return a;  
    return fac(n-1, n*a); }
```

Let us calculate 5!

return fac (5,1)

fac(5,1) = return fac(4, 5); a=5

fac(4,5) = return fac (3, 20); a=20

fac(3,20)= return fac (2,60); a=60

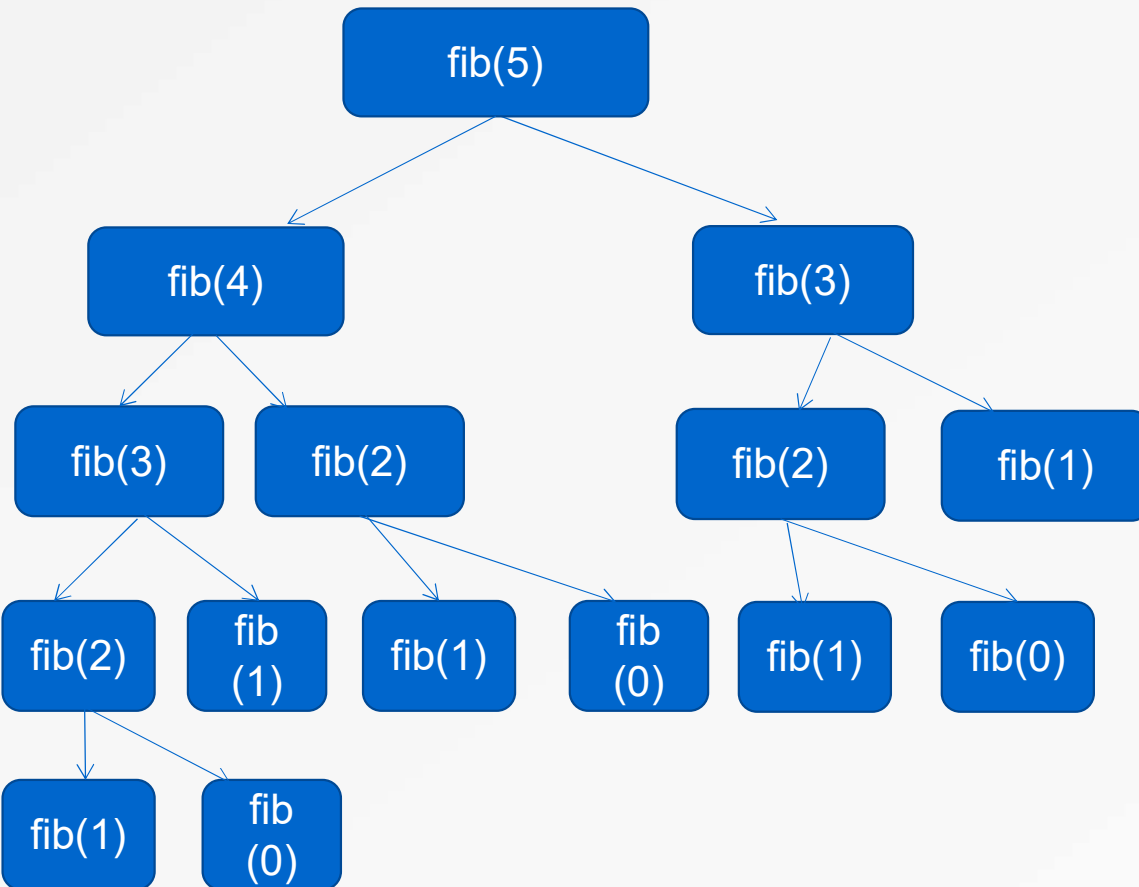
fac(2,60) = return fac(1, 120) ; a=120

fac(1,120) = return fac(0, 120) ; a=120

fac(0, 120) = return a, i.e.,120

- The general case of the transformation employs conversion to what is known as continuation-passing style.
- In effect, a recursive function can always avoid doing any work after returning from a recursive call by passing that work into the recursive call, in the form of a continuation.

Approach-I: Using Tree Recursion



```
int fib (int n)
{
    if (n==0)
        return 0;
    else if (n==1)
        return 1;
    else
        return (fib(n-1),
                fib(n-2))
}
```

Ex. fib(100)

How many times
fib (95) may have to
be calculated?

Complexity?

-Exponential

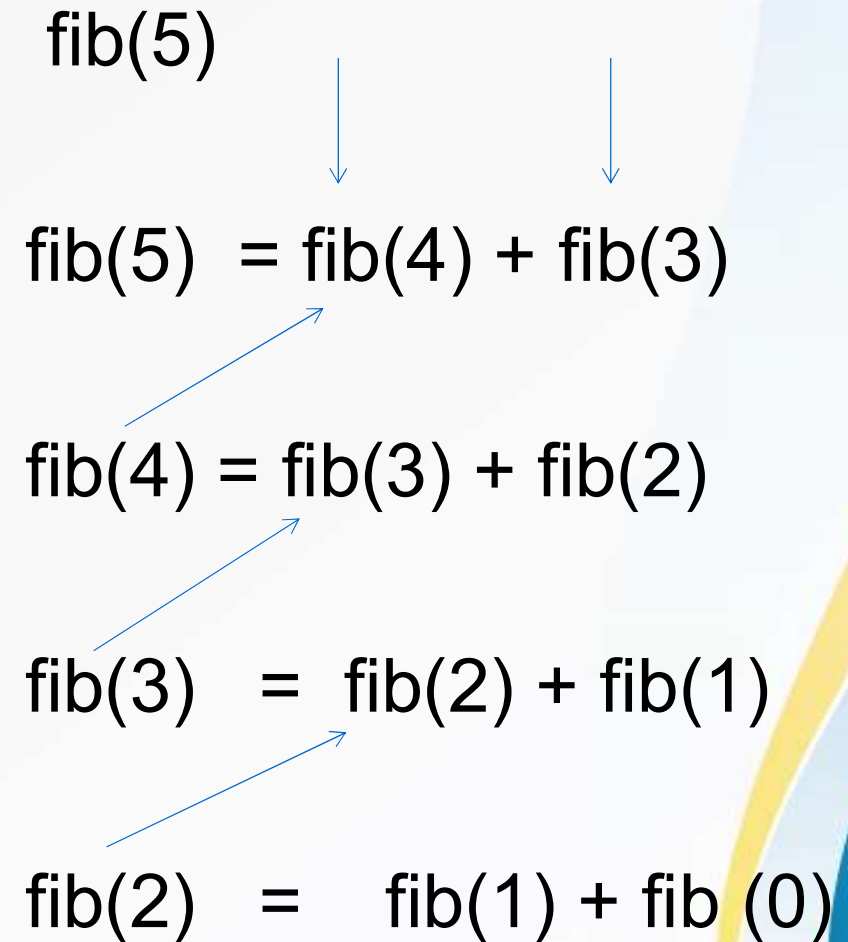
Iterative version -
o(n) (linear)

Tree recursion-one fun call leading to two recursive calls

Approach-II: Memoization

- Instead of repeatedly computing the same thing, we can store them in an auxiliary memory.

```
int fib_values[200]= {-1};  
int fib(int n){  
    if (fib_values[n]!=-1) return (fib_values[n]);  
    else if (n=0) { fib_values[0] = 0; return 0;}  
    else if (n==1) { fib_values[1]=1; return 1;}  
    else { fib_values[n] = fib(n-1) + fib(n-2);  
          return (fib_values[n]);}  
}
```



Approach-II:Memoization

- An optimization technique used primarily to speed up computer programs by storing the results of expensive function calls and returning the cached result when the same inputs occur again.
- Not a clean code.
- Array is used as a global array.
- We are trading space with time here.
- Trying to decrease the time, but space is increased.

Approach-III:Using Static

```
• int fib(int n)
{
    static int fib_values[n];
    // fib_values is stored in data
    segment in the memory
    // rest is same code.
}
```

- We can use static array inside the function instead of a global array.
- Values persist across the function calls.

Using Tail Recursive version of Fibo. Series

```
• int fibonacci (int n)
{
    return fib(n, 0, 1)
}
```

```
int fib(int n, int t1, int t2)
{
    if (n==0) return t1;
    else if (n==1) return t2;
    else return (fib(n-1, t2, t1+t2));
}
```

fib(1,3,5)

fib(2,2,3)

fib(3,1,2)

fib(4,1,1)

fib(5,0,1)

fibonacci(5)

5

- No space required for array

- No extra space for an activation record

- Same complexity as that of iterative version of fib series

Compiler's role for Tail recursive function

- Smart compilers written for functional languages make these transformations of recursive to tail-recursive of their own.
- General Steps
 - Try to make the program tail recursive
 - If not possible, try to make use of Memoization
 - If not, then use recursion

Stack Smashing/ Buffer Overflow

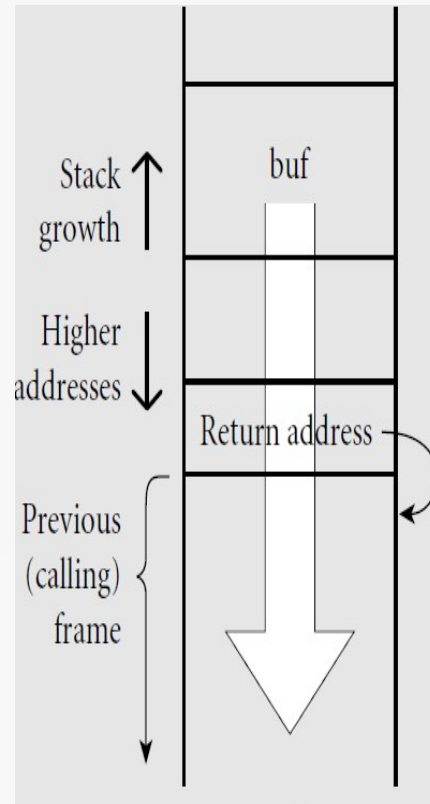
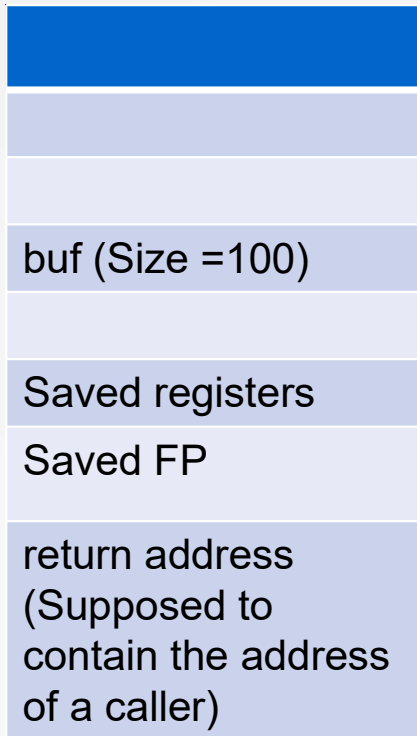
```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
int get_accnt_num(FILE *s)
{
    char buf[100];
    char *p = buf;
    do{
        *p = getc(s);
    }while(*p++!='\n');
    *p = '\0';
    return (atoi(buf));
}
```

```
main()
{
    FILE *fp;
    int r;
    fp = fopen("xyz.txt", "r");
    r = get_accnt_num(fp);
    printf("The no. of bytes read is
    %d\n", r);
}
```

Stack Smashing/ Buffer Overflow

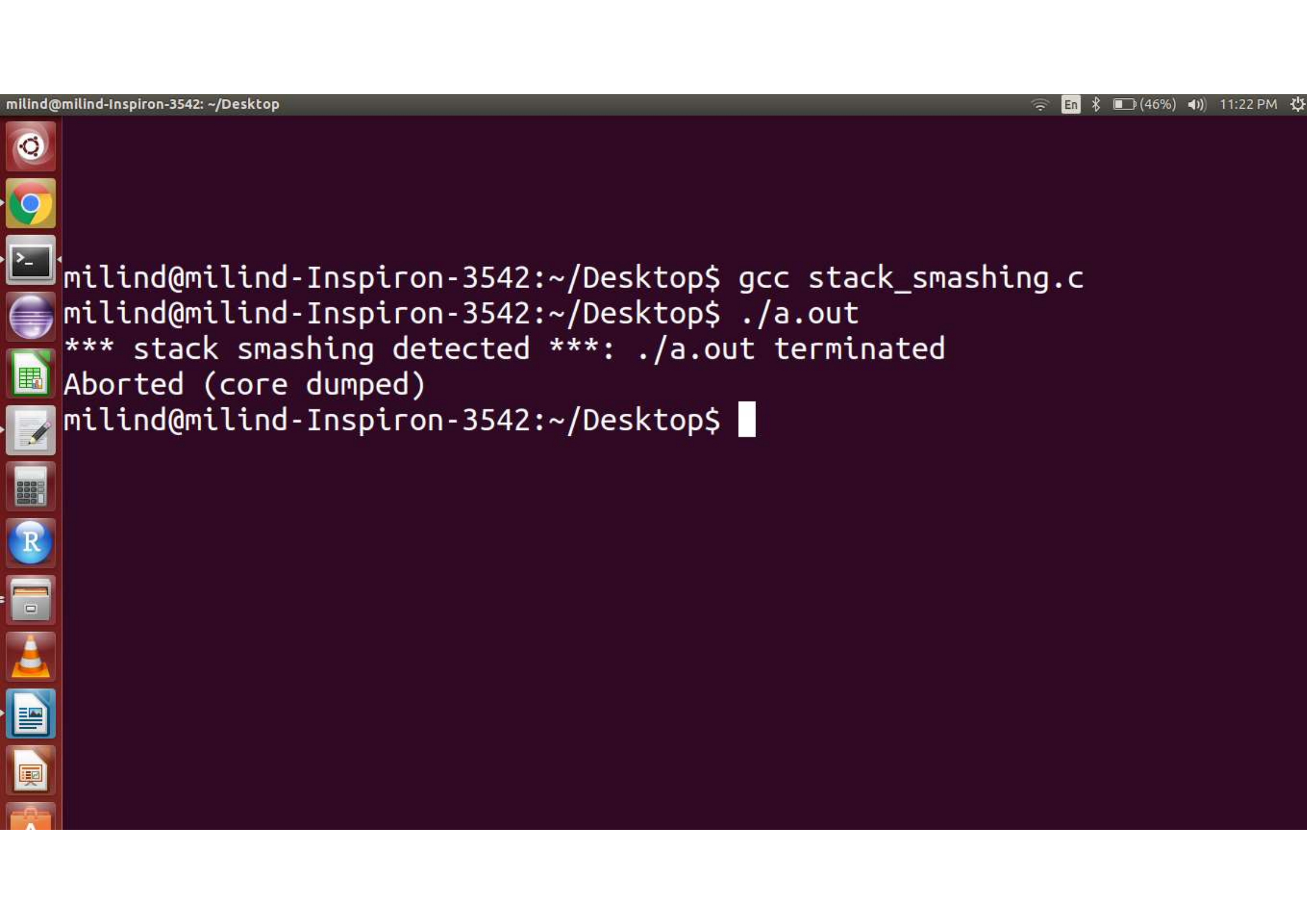
- If a process is trying to access/write memory beyond the storage space allocated for the function's local variables (which does not belong to your process) , then the stack smashing or stack buffer overflow is said to have occurred.
- If the file xyz.txt contains more than 100 characters, then this problem will occur in C/C++ as these languages do not check array bounds in order to speed up the program but are insecure.
- The language implementation must do this checking at run time.

Stack Smashing Cont..



- File with characters > 100, an array will overflow, buffer will overwrite p.
- If this is done further, then a clever programmer can replace return address with his/her own address to get the control of the system.

A typical stack frame

A terminal window with a dark purple background and a vertical dock on the left. The dock contains icons for various applications: a red circle with a white gear, a Google Chrome icon, a terminal icon, a purple sphere with white lines, a green spreadsheet icon, a document icon, a calculator icon, a blue circle with a white 'R', a folder icon, a traffic cone icon, a document with a magnifying glass icon, and a briefcase icon. The terminal text shows the compilation and execution of a program that triggers a stack smashing error.

```
milind@milind-Inspiron-3542:~/Desktop$ gcc stack_smashing.c
```

```
milind@milind-Inspiron-3542:~/Desktop$ ./a.out
```

```
*** stack smashing detected ***: ./a.out terminated
```

```
Aborted (core dumped)
```

```
milind@milind-Inspiron-3542:~/Desktop$
```

References

- Programming Language Pragmatics by Michael L. Scott, 3rd Edition (Elsevier)